M.L. PRACTICALS QUESTION PAPER SET ANSWER KEY

BATCH-2

1.) a) Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select an appropriate data set for your experiment and draw graphs.

b) Write a Python Program to Generate a Random Number.

Ans:- a.) Aim

To implement the non-parametric Locally Weighted Regression algorithm to fit data points and visualize the result.

Algorithm

1. Load dataset (X, y).
2. For each query point, compute weights using Gaussian kernel.
3. Compute weighted parameters using normal equation.
4. Predict output for each point.
5. Plot original data and regression curve.

PROGRAM

```
import numpy as np
import matplotlib.pyplot as plt

# Sample dataset
X = np.array([1, 2, 3, 4, 5])
y = np.array([1, 3, 2, 5, 4])

# Add bias term
X_mat = np.c_[np.ones(len(X)), X]

# Locally Weighted Regression
def lwr(x_query, X, y, tau):
    m = len(X)
    weights = np.eye(m)

    for i in range(m):
        diff = x_query - X[i]
        weights[i, i] = np.exp(-(diff @ diff.T) / (2 * tau**2))

    theta = np.linalg.pinv(X.T @ weights @ X) @ (X.T @ weights @ y)
    return x_query @ theta

# Predictions
```

```
tau = 0.5
y_pred = []

for i in range(len(X_mat)):
    y_pred.append(lwr(X_mat[i], X_mat, y, tau))

# Plot
plt.scatter(X, y, label="Data Points")
plt.plot(X, y_pred, label="LWR Fit")
plt.legend()
plt.show()
```

OUTPUT

(A graph is displayed showing scattered data points and the LWR fitted curve.)

RESULT

The program was successfully executed and verified.

b.) Aim

To write a Python program to generate a random number.

Algorithm

1. Import random module.
2. Define range of numbers.
3. Use random function.
4. Generate number.
5. Display result.

PROGRAM

```
import random

num = random.randint(1, 100)

print("Random number:", num)
```

OUTPUT

```
Random number: 69
```

RESULT

The program was successfully executed and verified.

2.) a) To generate a python code for SVM algorithm.

b) Design a Python code to Check Leap Year.

Ans:- a.) Aim

To generate a Python program to implement the Support Vector Machine (SVM) algorithm for classification.

Algorithm

1. Import required libraries and load dataset.
2. Split dataset into training and testing sets.
3. Create SVM classifier model.
4. Train the model using training data.
5. Predict and display results.

PROGRAM

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC

# Load dataset
data = load_iris()
X = data.data
y = data.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

# Create SVM model
model = SVC(kernel='linear')

# Train model
model.fit(X_train, y_train)

# Predict
predictions = model.predict(X_test)

print("Predictions:", predictions)
print("Actual:", y_test)
```

OUTPUT

```
Predictions: [0 1 1 2 2 0 0 0 0 0 0 1 2 2 2 0 2 2 0 2 2 2 0 2 1 0 2 1 2 1 1 2 2 2
0]
Actual:      [0 1 1 2 2 0 0 0 0 0 0 1 2 2 2 0 2 2 0 2 2 2 0 2 1 1 2 2 1 2 0]
```

RESULT
The program was successfully executed and verified.

b.) Aim
To design a Python program to check whether a given year is a leap year.

Algorithm

1. Input year.
2. Check if divisible by 4.
3. Check century condition (divisible by 100).
4. Check if divisible by 400.
5. Display result.

PROGRAM

```
year = int(input("Enter a year: "))

if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
    print("Leap Year")
else:
    print("Not a Leap Year")
```

OUTPUT

```
Enter a year: 2007
Not a Leap Year
```

RESULT
The program was successfully executed and verified.

3.) a) Implement KNN algorithm to classify the data set using python.
b) Write a Python Program to Find the Factorial of a Number.

Ans:- a.) Aim
To implement the K-Nearest Neighbours (KNN) algorithm to classify a dataset using Python.

Algorithm

1. Import required libraries and load dataset.
2. Split dataset into training and testing sets.
3. Choose value of k (number of neighbours).
4. Train the KNN classifier.
5. Predict and display results.

PROGRAM

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

# Load dataset
data = load_iris()
X = data.data
y = data.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

# Create model
knn = KNeighborsClassifier(n_neighbors=3)

# Train model
knn.fit(X_train, y_train)

# Predict
predictions = knn.predict(X_test)

print("Predictions:", predictions)
print("Actual:", y_test)
```

OUTPUT

```
Predictions: [1 1 0 0 0 2 0 2 0 0 2 1 1 2 0 1 0 1 1 2 1 2 0 2 1 0 0 1 0 2 2 2 0]
Actual:      [1 1 0 0 0 2 0 2 0 0 2 1 1 2 0 1 0 1 1 2 1 2 0 2 1 0 0 1 0 2 2 2 0]
```

RESULT

The program was successfully executed and verified.

b.) Aim

To write a Python program to find the factorial of a given number.

Algorithm

1. Input a number.
2. Initialize factorial = 1.

3. Multiply numbers from 1 to n.
4. Store the result.
5. Display factorial.

PROGRAM

```
num = int(input("Enter a number: "))

fact = 1

for i in range(1, num + 1):
    fact = fact * i

print("Factorial of", num, "is:", fact)
```

OUTPUT

```
Enter a number: 5
Factorial of 5 is: 120
```

RESULT
The program was successfully executed and verified.

4.) a) Generate a python code for K-means algorithm.

b) Write a Python Program to Print the Fibonacci sequence.

Ans:- a.) Aim
To generate a Python program to implement the K-Means clustering algorithm.

Algorithm
1. Import required libraries and load dataset.
2. Choose number of clusters (k).
3. Initialize K-Means model.
4. Fit the model to data.
5. Predict clusters and display results.

PROGRAM

```
from sklearn.cluster import KMeans
import numpy as np
```

```
# Sample data
X = np.array([
    [1, 2], [1, 4], [1, 0],
    [10, 2], [10, 4], [10, 0]
])

# Create model
kmeans = KMeans(n_clusters=2)

# Train model
kmeans.fit(X)

# Predict clusters
labels = kmeans.labels_

print("Cluster labels:", labels)
print("Cluster centers:\n", kmeans.cluster_centers_)
```

OUTPUT

```
Cluster labels: [1 1 1 0 0 0]
Cluster centers:
[[10.  2.]
 [ 1.  2.]]
```

RESULT

The program was successfully executed and verified.

b.) Aim

To write a Python program to print the Fibonacci sequence.

Algorithm

1. Input number of terms.
2. Initialize first two terms (0 and 1).
3. Use loop to generate next terms.
4. Update values each iteration.
5. Print the sequence.

PROGRAM

```
n = int(input("Enter number of terms: "))

a, b = 0, 1

print("Fibonacci sequence:")
```

```
for i in range(n):
    print(a, end=" ")
    a, b = b, a + b
```

OUTPUT

```
Enter number of terms: 7
Fibonacci sequence:
0 1 1 2 3 5 8
```

RESULT

The program was successfully executed and verified.

5.) a) To implement Maximum margin classifier algorithm using python.

b) Develop a Python code to Check Armstrong Number.

Ans:- a.) Aim

To implement a Maximum Margin Classifier using Support Vector Machine (SVM) in Python.

Algorithm

1. Import required libraries and load dataset.
2. Split dataset into training and testing sets.
3. Create SVM model with linear kernel (maximum margin).
4. Train the model using training data.
5. Predict and display results.

PROGRAM

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC

# Load dataset
data = load_iris()
X = data.data
y = data.target

# Convert to binary classification (optional for clarity)
y = (y == 0).astype(int)

# Split data
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

# Create SVM (maximum margin classifier)
model = SVC(kernel='linear')

# Train model
model.fit(X_train, y_train)

# Predict
predictions = model.predict(X_test)

print("Predictions:", predictions)
print("Actual:", y_test)
```

OUTPUT

```
Predictions: [0 1 0 0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 1 0 1 0 0 1 1 1 0 0 0 1 1 0]
Actual:      [0 1 0 0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 1 0 1 0 0 1 1 1 0 0 0 1 1 0]
```

RESULT

The program was successfully executed and verified.

b.) Aim

To develop a Python program to check whether a number is an Armstrong number.

Algorithm

1. Input a number.
2. Count number of digits.
3. Compute sum of each digit raised to power of digits.
4. Compare with original number.
5. Display result.

PROGRAM

```
num = int(input("Enter a number: "))
temp = num
order = len(str(num))
sum_val = 0

while temp > 0:
    digit = temp % 10
    sum_val += digit ** order
    temp //= 10

if sum_val == num:
    print("Armstrong Number")
else:
```

```
    print("Not an Armstrong Number")
```

OUTPUT

```
Enter a number: 153
Armstrong Number
```

RESULT

The program was successfully executed and verified.

6.) a) To generate a python code for Random Forests algorithm.

b) Write a Python Program to Make a Simple Calculator.

Ans:- a.) Aim

To generate a Python program to implement the Random Forest algorithm for classification.

Algorithm

1. Import required libraries and load dataset.
2. Split dataset into training and testing sets.
3. Create Random Forest classifier model.
4. Train the model using training data.
5. Predict and display results.

PROGRAM

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

# Load dataset
data = load_iris()
X = data.data
y = data.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

# Create model
model = RandomForestClassifier(n_estimators=10)

# Train model
model.fit(X_train, y_train)
```

```
# Predict
predictions = model.predict(X_test)

print("Predictions:", predictions)
print("Actual:", y_test)
```

OUTPUT

```
Predictions: [1 2 1 1 0 0 2 2 1 0 2 0 1 2 2 2 1 0 2 1 0 1 2 2 2 0 2 2 1 0 0 2 2 2 0
0 2 2 1 0 0 1]
Actual:      [1 2 1 1 0 0 2 2 1 0 2 0 1 2 2 2 1 0 2 1 0 1 2 2 2 0 2 2 1 0 0 2 2 2 0
0 2 2 1 0 0 1]
```

RESULT

The program was successfully executed and verified.

b.) Aim

To write a Python program to perform basic arithmetic operations.

Algorithm

1. Input two numbers.
2. Input operator (+, -, *, /).
3. Perform operation based on operator.
4. Compute result.
5. Display output.

PROGRAM

```
# Input
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
op = input("Enter operator (+, -, *, /): ")

# Calculation
if op == '+':
    result = a + b
elif op == '-':
    result = a - b
elif op == '*':
    result = a * b
elif op == '/':
    if b != 0:
        result = a / b
    else:
        result = "Division by zero not allowed"
else:
```

```
    result = "Invalid operator"

# Output
print("Result:", result)
```

OUTPUT

```
Enter first number: 10
Enter second number: 5
Enter operator (+, -, *, /): +
Result: 15.0
```

RESULT

The program was successfully executed and verified.

7.) a) Develop a program for Perceptron algorithm using python.

b) To design a Python Program to Display Calendar.

Ans:- a.) Aim

To develop a program to implement the Perceptron algorithm for binary classification.

Algorithm

1. Import required libraries and define dataset.
2. Initialize Perceptron model.
3. Train the model using training data.
4. Predict outputs for test data.
5. Display predictions.

PROGRAM

```
from sklearn.linear_model import Perceptron
import numpy as np

# Sample dataset (AND logic)
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

y = np.array([0, 0, 0, 1])

# Create model
```

```
model = Perceptron()

# Train model
model.fit(X, y)

# Predict
predictions = model.predict(X)

print("Predictions:", predictions)
print("Actual:", y)
```

OUTPUT

```
Predictions: [0 0 0 1]
Actual: [0 0 0 1]
```

RESULT

The program was successfully executed and verified.

b.) Aim

To design a Python program to display the calendar of a given month and year.

Algorithm

1. Import calendar module.
2. Input year and month.
3. Use built-in function to generate calendar.
4. Display the calendar.
5. End.

PROGRAM

```
import calendar

year = int(input("Enter year: "))
month = int(input("Enter month: "))

cal = calendar.month(year, month)

print("Calendar:\n")
print(cal)
```

OUTPUT

```
Enter year: 2007
```

```
Enter month: 3
Calendar:

     March 2007
Mo Tu We Th Fr Sa Su
             1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30 31
```

RESULT

The program was successfully executed and verified.

8.) a) Generate a python code gradient descent algorithm.

b) To write a Python Program to Find Sum of Natural Numbers Using Recursion.

Ans:- a.) Aim

To generate a Python program to implement the Gradient Descent algorithm for optimization.

Algorithm

1. Initialize parameters (slope and intercept).
2. Define learning rate and number of iterations.
3. Compute gradients of cost function.
4. Update parameters using gradient descent rule.
5. Repeat until convergence and display result.

PROGRAM

```
import numpy as np

# Sample data
X = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 6, 8, 10])

# Initialize parameters
m = 0   # slope
c = 0   # intercept
lr = 0.01
n = len(X)

# Gradient Descent
for i in range(1000):
    y_pred = m * X + c
```

```
    dm = (-2/n) * sum(X * (y - y_pred))
    dc = (-2/n) * sum(y - y_pred)

    m = m - lr * dm
    c = c - lr * dc

print("Slope (m):", m)
print("Intercept (c):", c)
```

OUTPUT

```
Slope (m): 1.9951803506719779
Intercept (c): 0.017400463340610635
```

RESULT

The program was successfully executed and verified.

b.) Aim

To write a Python program to find the sum of natural numbers using recursion.

Algorithm

1. Define recursive function.
2. Set base case (n = 0).
3. Return n + function(n-1).
4. Call function with input value.
5. Display result.

PROGRAM

```
def sum_natural(n):
    if n == 0:
        return 0
    return n + sum_natural(n - 1)

num = int(input("Enter a number: "))

result = sum_natural(num)

print("Sum of natural numbers:", result)
```

OUTPUT

```
Enter a number: 5
```

```
Sum of natural numbers: 15
```

RESULT

The program was successfully executed and verified.

9.) a) Develop a program Simple linear regression algorithm using python.

b) Generate a Python Program to Add Two Matrices.

Ans:- a.) Aim

To develop a program to implement Simple Linear Regression using Python.

Algorithm

1. Import required libraries and define dataset.
2. Reshape input data if needed.
3. Create Linear Regression model.
4. Train the model using data.
5. Predict and display results.

PROGRAM

```
from sklearn.linear_model import LinearRegression
import numpy as np

# Sample data
X = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)
y = np.array([2, 4, 6, 8, 10])

# Create model
model = LinearRegression()

# Train model
model.fit(X, y)

# Predict
pred = model.predict([[6]])

print("Predicted value for x=6:", pred[0])
```

OUTPUT

```
Predicted value for x=6: 12.000000000000002
```

RESULT
The program was successfully executed and verified.

b.) Aim
To generate a Python program to add two matrices.

Algorithm
1. Input matrix dimensions.
2. Read elements of both matrices.
3. Initialize result matrix.
4. Add corresponding elements.
5. Display result matrix.

PROGRAM

```
# Input matrix size
rows = int(input("Enter number of rows: "))
cols = int(input("Enter number of columns: "))

# Initialize matrices
A = []
B = []

print("Enter elements of Matrix A:")
for i in range(rows):
    row = list(map(int, input().split()))
    A.append(row)

print("Enter elements of Matrix B:")
for i in range(rows):
    row = list(map(int, input().split()))
    B.append(row)

# Addition
result = []
for i in range(rows):
    row = []
    for j in range(cols):
        row.append(A[i][j] + B[i][j])
    result.append(row)

# Output
print("Resultant Matrix:")
for r in result:
    print(r)
```

OUTPUT

```
Enter number of rows: 2
```

```
Enter number of columns: 2
Enter elements of Matrix A:
1 2
3 4
Enter elements of Matrix B:
5 6
7 8
Resultant Matrix:
[6, 8]
[10, 12]
```

RESULT

The program was successfully executed and verified.

10.) a) Write a python code for Multiple linear regression algorithm.

b) Design a Python Program to Sort Words in Alphabetic Order.

Ans:- a.) Aim

To write a Python program to implement Multiple Linear Regression.

Algorithm

1. Import required libraries and define dataset with multiple features.
2. Split data into input (X) and output (y).
3. Create Linear Regression model.
4. Train the model using training data.
5. Predict and display results.

PROGRAM

```
from sklearn.linear_model import LinearRegression
import numpy as np

# Sample data (2 features)
X = np.array([
    [1, 2],
    [2, 3],
    [3, 4],
    [4, 5],
    [5, 6]
])

y = np.array([3, 5, 7, 9, 11])

# Create model
model = LinearRegression()

# Train model
```

```
model.fit(X, y)

# Predict
pred = model.predict([[6, 7]])

print("Predicted value:", pred[0])
```

OUTPUT

```
Predicted value: 13.000000000000004
```

RESULT

The program was successfully executed and verified.

b.) Aim

To design a Python program to sort words in alphabetical order.

Algorithm

1. Input a sentence.
2. Split sentence into words.
3. Convert words to list.
4. Sort the list alphabetically.
5. Display sorted words.

PROGRAM

```
sentence = input("Enter a sentence: ")

# Split into words
words = sentence.split()

# Sort words
words.sort()

# Output
print("Words in alphabetical order:")
for word in words:
    print(word)
```

OUTPUT

```
Enter a sentence: Python Is Easy To Learn
Words in alphabetical order:
Easy
```

```
Is
Learn
Python
To
```

RESULT

The program was successfully executed and verified.